

Design Patterns

Reusable Solutions to Common Software Problems

Software Engineering Team

Department of Computer Science

May 5, 2026

Presentation Outline

- 1 Introduction to Design Patterns
- 2 Creational Patterns - Singleton
- 3 Creational Patterns - Factory Method
- 4 Structural Patterns - Adapter
- 5 Behavioral Patterns - Observer
- 6 Behavioral Patterns - Strategy

What Are Design Patterns?

Definition

A **design pattern** is a general, reusable solution to a commonly occurring problem in software design.

Key Characteristics

- **Proven** - Based on real-world experience
- **Reusable** - Can be applied to many situations
- **Documented** - Shared vocabulary for developers
- **Language-agnostic** - Works across programming languages

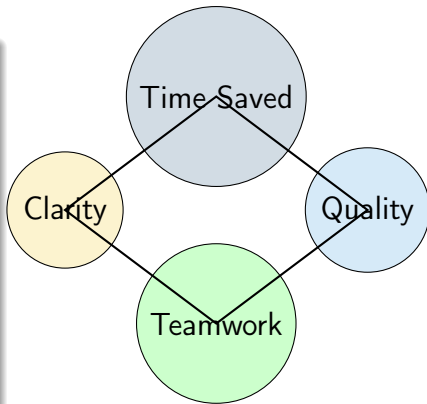
Not a Silver Bullet!

Patterns are templates, not finished code. Use them wisely.

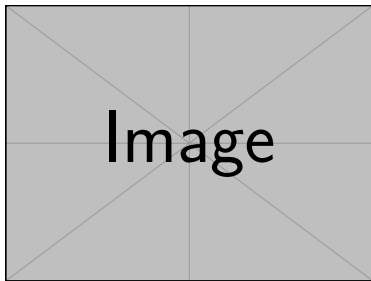
Why Are Design Patterns Important?

Benefits

- **Reusability** - Save time and effort
- **Communication** - "Use Singleton here"
- **Best Practices** - Decades of experience
- **Maintainability** - Modular, flexible code
- **Avoid Reinventing** - Learn from others



The Gang of Four (GoF)



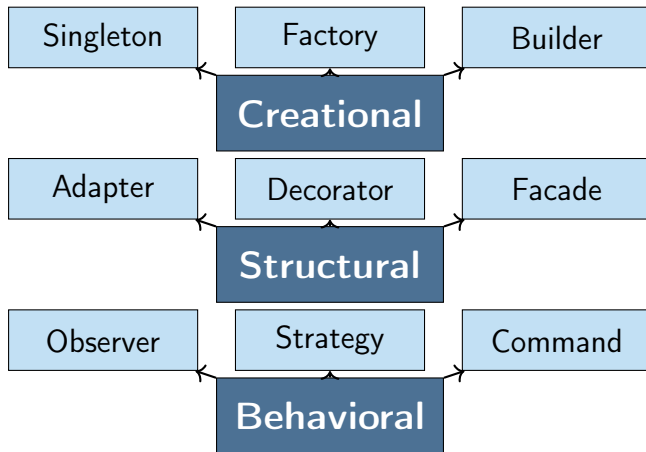
Gamma, Helm, Johnson, Vlissides

"Design Patterns: Elements of Reusable Object-Oriented Software" (1994)

23 Patterns documented in 3 categories:

- Creational (5)
- Structural (7)
- Behavioral (11)

Pattern Classification



Singleton Pattern

Definition

Ensures a class has **only one instance** and provides a **global point of access**.

When to Use

- Database connections
- Logging services
- Configuration managers
- Thread pools
- Cache managers

Warning

Overusing Singleton leads to:

- Hidden dependencies
- Testing difficulties
- Global state problems

Singleton - Real-World Analogy

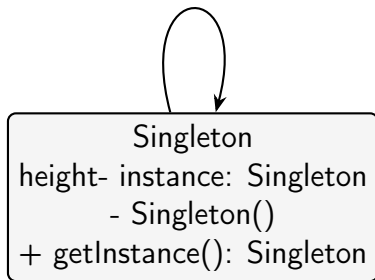
Government of a Country

Everyone accesses the SAME government

Only ONE official government at any time

Just like in a country, Singleton ensures only ONE instance exists!

Singleton - UML Diagram



Only ONE instance exists

Singleton - Dart/Flutter Example

Singleton in Dart

```
1 class DatabaseService {  
2   // Single instance  
3   static final DatabaseService _instance =  
4     DatabaseService._internal();  
5  
6   // Private constructor  
7   DatabaseService._internal() {  
8     print("Instance created!");  
9   }  
10  
11  // Factory returns same instance  
12  factory DatabaseService() {  
13    return _instance;  
14  }
```

Singleton - Python Example

Singleton in Python

```
1 class DatabaseConnection:
2     _instance = None
3
4     def __new__(cls):
5         if cls._instance is None:
6             print("Creating instance")
7             cls._instance = super().__new__(cls)
8             return cls._instance
9
10    def connect(self):
11        print("Connected to DB")
12
13    # Usage
14    db1 = DatabaseConnection()
```

Singleton - Pro Tips

Best Practices

- Use lazy initialization
- Make constructor private
- Provide static access method
- Consider thread-safety

In Flutter

Use `GetIt` package or `Provider` with `Singleton` for:

- API clients
- Repository classes
- BLoC/Cubit instances

Factory Method Pattern

Definition

Defines an interface for creating objects, but lets **subclasses decide which class to instantiate**.

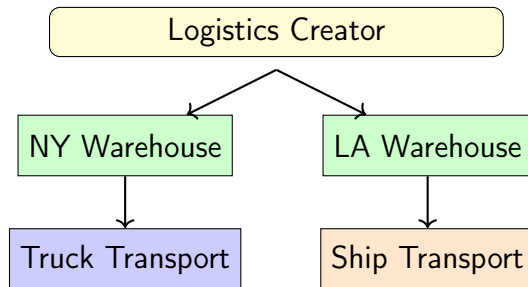
Problem Solved

Direct instantiation (`new`) couples code to concrete classes.

Benefits

- Loose coupling
- Open/Closed principle
- Better testability

Factory Method - Analogy



Each warehouse **creates** appropriate transport for its location

Factory Method - Python Example

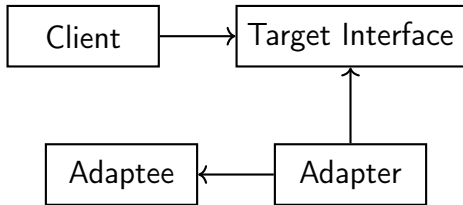
Factory Method in Python

```
1 from abc import ABC, abstractmethod
2
3 class Button(ABC):
4     @abstractmethod
5     def render(self): pass
6
7 class WindowsButton(Button):
8     def render(self): return "Windows button"
9
10 class MacButton(Button):
11     def render(self): return "Mac button"
12
13 class Dialog(ABC):
14     @abstractmethod
15     def create_button(self) -> Button: pass
16
```

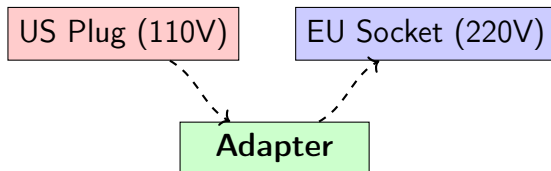
Adapter Pattern

Definition

Converts the interface of a class into another interface that clients expect.



Adapter - Real-World Example



Converts incompatible interfaces!

Adapter - Practical Code

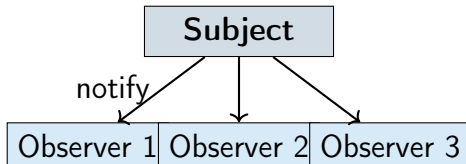
Adapter Pattern in Flutter

```
1 // Incompatible old API
2 class OldApiResponse {
3   String get userName => data['user_name'];
4 }
5
6 // Our expected interface
7 abstract class UserDataSource {
8   Future<User> getUser(String id);
9 }
10
11 // Adapter makes old API work with our system
12 class OldApiAdapter implements UserDataSource {
13   @override
14   Future<User> getUser(String id) async {
15     final oldData = await fetchOldApi(id);
16     return User(
```

Observer Pattern

Definition

Defines a one-to-many dependency so that when one object changes state, all dependents are notified.



YouTuber posts → All subscribers get notified automatically!

Observer - Dart/Flutter with Streams

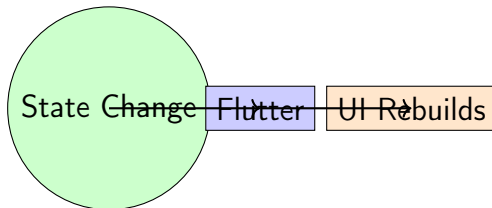
Observer Pattern in Flutter

```
1 import 'dart:async';
2
3 class NewsAgency {
4   final _controller = StreamController<String>();
5   Stream<String> get news => _controller.stream;
6
7   void publish(String news) {
8     _controller.add(news); // Notify all!
9   }
10 }
11
12 // Subscribe and react
13 void main() {
14   final agency = NewsAgency();
15
16   agency.news.listen((news) {
```

Observer - Flutter's Built-in Usage

Flutter Uses Observer Pattern EVERYWHERE

- **setState()** - Notifies widget to rebuild
- **Provider** - Listens to model changes
- **BLoC** - Stream-based state management
- **StreamBuilder** - Reacts to stream updates
- **ValueNotifier** - Simple observable value



Strategy Pattern

Definition

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

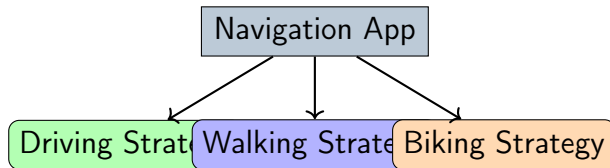
Problem Solved

- Massive if-else statements
- Code duplication
- Difficult to add new algorithms

Solution

- Extract algorithms to separate classes
- Client chooses which to use
- Easy to add new strategies

Strategy - Navigation Example



Each strategy calculates route DIFFERENTLY, but app calls SAME method

Strategy - Python Example

Strategy Pattern in Python

```
1 from abc import ABC, abstractmethod
2
3 class PaymentStrategy(ABC):
4     @abstractmethod
5     def pay(self, amount): pass
6
7 class CreditCardPayment(PaymentStrategy):
8     def pay(self, amount):
9         print(f"        Paid ${amount} via Credit Card")
10
11 class PayPalPayment(PaymentStrategy):
12     def pay(self, amount):
13         print(f"        Paid ${amount} via PayPal")
14
15 class ShoppingCart:
16     def init_(self):
```

Pattern Decision Matrix

Pattern	Use When	Avoid When	Flutter Example
Singleton	Exactly one instance needed	Testing is critical	DatabaseService, ApiClient
Factory Method	Class can't know exact type	Simple creation works	Theme-aware widgets
Adapter	Incompatible interfaces	You control all code	API response conversion
Observer	One-to-many notification needed	Simple direct calls work	Provider, BLoC, Stream-Builder
Strategy	Multiple algorithms interchangeable	Only 1-2 variants	Payment methods, validators

Common Anti-Patterns to Avoid

The Golden Hammer

Using the same pattern everywhere, even when simple code works.

Over-Engineering

Factory + Strategy + Observer for a simple counter app.

Cargo Cult

Implementing patterns without understanding the problem.

Singleton Madness

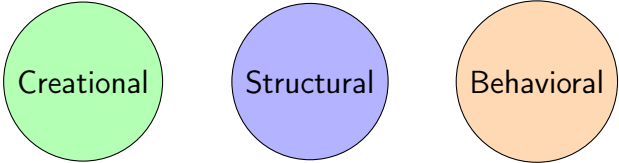
Creating singletons for everything = global state spaghetti.

How to Choose and Use Patterns

- ➊ **Identify what varies** - Find what changes in your system
- ➋ **Encapsulate the variation** - Hide it behind an interface
- ➌ **Start simple** - Implement simplest solution first
- ➍ **Refactor to pattern** - Only add pattern when needed
- ➎ **Favor composition over inheritance**
- ➏ **Program to interfaces, not implementations**

Key Takeaways

Design Patterns are REUSABLE SOLUTIONS to COMMON PROBLEMS



Creational

Structural

Behavioral

Learn the PROBLEM first, then the PATTERN

Resources for Further Learning

Books

- Design Patterns (GoF)
- Head First Design Patterns
- Clean Code (Robert Martin)

Practice Platforms

- LeetCode (Design Problems)
- Exercism.io
- Flutter Design Patterns on GitHub

Websites

- Refactoring Guru
- SourceMaking
- Dart Design Patterns

Try These!

- ❶ **Singleton:** Thread-safe logger in Python/Dart
- ❷ **Factory:** Document generator (PDF, Word, HTML)
- ❸ **Adapter:** Integrate two different weather APIs
- ❹ **Observer:** Stock price notifier with displays
- ❺ **Strategy:** Sorting app (bubble, quick, merge sort)

"A pattern is a reusable solution to a recurring problem in a specific context."

Thank You!

Questions?

Happy Coding!